

**ANO
2026**



UNINTER

**CADERNO DE RESPOSTAS DA ATIVIDADE
PRÁTICA DE:
Compiladores**

ALUNO: Vinicius Gonçalves – 4471599

Prof. Diógenes Cogo Furlan, Me.

TESTE NEGATIVO (EXEMPLOS COM ERROS)

Faça o teste para 3 programas diferentes

- O primeiro com erro léxico
- O segundo com erro sintático
- O terceiro com erro semântico

Exemplo de Entrada com Erro Léxico em MINI-LISP (PROGRAMA 1):
(print @)

Apresentar a captura de tela evidenciando a mensagem de erro:

```
Erro detectado corretamente:  
Caractere inesperado '@' na linha 2, coluna 13
```

Exemplo de Entrada com Erro Sintático em MINI-LISP (PROGRAMA 2):
(print 10 20)

Apresentar a captura de tela evidenciando a mensagem de erro:

```
Erro detectado corretamente:  
Esperado ')'.  
          ^
```

Exemplo de Entrada com Erro Semântico em MINI-LISP (PROGRAMA 3):
(print x)

Apresentar a captura de tela evidenciando a mensagem de erro:

```
Erro detectado corretamente:  
Erro semântico: variável 'x' não declarada.
```

TESTE POSITIVO (EXEMPLOS CORRETOS)

Faça o teste para 1 ou 2 programas diferentes

- Um programa básico, só com operações e prints
- Um programa com seleção e/ou repetição

Exemplo de Entrada correta em MINI-LISP (PROGRAMA 1):

```
(begin
  (print 10)
  (print (+ 10 20))
  (print (- 100 30))
)
```

Lista de tokens/lexemas:


```
(begin
  (set contador 1)
  (while (<= contador 5)
    (begin
      (if (>= contador 3)
        (print (* contador 10))
        (print contador)
      )
      (set contador (+ contador 1))
    )
  )
)
```

Lista de tokens/lexemas:

```
1 LPAREN (
2 IDENTIFIER begin
3 LPAREN (
4 IDENTIFIER set
5 IDENTIFIER contador
6 NUMBER 1
7 RPAREN )
8 LPAREN (
9 IDENTIFIER while
10 LPAREN (
11 LESS_EQUAL <=
12 IDENTIFIER contador
13 NUMBER 5
14 RPAREN )
15 LPAREN (
16 IDENTIFIER begin
17 LPAREN (
18 IDENTIFIER if
19 LPAREN (
20 GREATER_EQUAL >=
21 IDENTIFIER contador
22 NUMBER 3
23 RPAREN )
24 LPAREN (
25 IDENTIFIER print
26 LPAREN (
27 MULTIPLY *
28 IDENTIFIER contador
29 NUMBER 10
30 RPAREN )
31 RPAREN )
32 LPAREN (
33 IDENTIFIER print
34 IDENTIFIER contador
35 RPAREN )
36 RPAREN )
37 LPAREN (
38 IDENTIFIER set
39 IDENTIFIER contador
40 LPAREN (
41 PLUS +
42 IDENTIFIER contador
43 NUMBER 1
44 RPAREN )
45 RPAREN )
46 RPAREN )
47 RPAREN )
48 RPAREN )
49 EOF
```

Tabela de Símbolos:

Identificador	Endereço MEPA	Tipo Presumido	Escopo
contador	0	number	global

Código MEPA:

```
CRCT 1
ARMZ contador
L0:
CRVL contador
CRCT 5
CMEG
DSVF L1
CRVL contador
CRCT 3
CMAG
DSVF L2
CRVL contador
CRCT 10
MULT
IMPR
DSVS L3
L2:
CRVL contador
IMPR
L3:
CRVL contador
CRCT 1
SOMA
ARMZ contador
DSVS L0
L1:
```

CÓDIGO FONTE

Insira seu código-fonte do compilador abaixo.

- Separe os arquivos, caso exista vários arquivos fontes
- Não é necessário que o programa seja realizado em vários arquivos
- O código-fonte deve estar indentado e tabulado
- Não precisa ficar colorido como na IDE

Apresentação do código-fonte:

Separar os arquivos caso sejam feitos vários arquivos fonte, nomeando cada um:

main.py

```
def main():
    test_lexer()
    test_lexer_negative()

    test_parser()
    test_parser_negative()

    test_semantic()
    test_semantic_negative()

if __name__ == "__main__":
    main()

source = ""
(begin

    (set contador 1)

    (while (<= contador 5)

        (begin
```



```

        (if (>= contador 3)
          (print (* contador 10))
          (print contador)
        )

        (set contador (+ contador 1))

      )
    )
)
"""

lexer = Lexer(source)

tokens = lexer.tokenize()

parser = Parser(tokens)

expressions = parser.parse()

semantic_analyzer = SemanticAnalyzer()

semantic_analyzer.analyze(
    expressions
)

generator = MEPA(
    semantic_analyzer.symbol_table
)

code = generator.generate(
    expressions
)

Report.print_tokens(tokens)

Report.print_symbols(
    semantic_analyzer.symbol_table
)

Report.print_code(code)

```

parser.py

```

class Parser:

    def __init__(self, tokens):

```

```
self.tokens = tokens
self.current = 0

def parse(self):
    expressions = []

    while not self.is_at_end():
        expressions.append(
            self.expression()
        )

    return expressions

def expression(self):
    token = self.peak()

    if token.type == TokenType.NUMBER:
        return self.number()

    if token.type == TokenType.IDENTIFIER:
        return self.identifier()

    if token.type == TokenType.LPAREN:
        return self.list()

    raise Exception(
        f"Token inesperado: {token}"
    )

def number(self):
    token = self.advance()

    return NumberExpression(
        int(token.lexeme)
    )

def identifier(self):
    token = self.advance()

    return IdentifierExpression(
        token.lexeme
    )

def list(self):
    self.consume(
        TokenType.LPAREN,
```

```
        "Esperado '('"
    )

    if self.check(TokenType.RPAREN):
        raise Exception(
            "Lista vazia não é permitida."
        )

    operator = self.advance()

    if operator.type in (
        TokenType.PLUS,
        TokenType.MINUS,
        TokenType.MULTIPLY,
        TokenType.DIVIDE,
    ):
        return self.binary_expression(
            operator
        )

    if operator.type in (
        TokenType.GREATER_EQUAL,
        TokenType.GREATER_THAN,
        TokenType.LESS_EQUAL,
        TokenType.LESS_THAN,
    ):
        return self.binary_expression(
            operator
        )

    if operator.type == TokenType.IDENTIFIER:

        if operator.lexeme == "print":
            return self.print_expression()

        if operator.lexeme == "set":
            return self.set_expression()

        if operator.lexeme == "begin":
            return self.begin_expression()

        if operator.lexeme == "if":
            return self.if_expression()

        if operator.lexeme == "while":
            return self.while_expression()
```

```
    raise Exception(
        f"Operador ou forma especial desconhecida: "
        f"{operator.lexeme}"
    )

def binary_expression(self, operator):

    left = self.expression()

    right = self.expression()

    self.consume(
        TokenType.RPAREN,
        "Esperado ')".
    )

    return BinaryExpression(
        operator.lexeme,
        left,
        right
    )

def print_expression(self):

    value = self.expression()

    self.consume(
        TokenType.RPAREN,
        "Esperado ')".
    )

    return PrintExpression(
        value
    )

def set_expression(self):

    name = self.consume(
        TokenType.IDENTIFIER,
        "Esperado identificador após 'set'.".
    )

    value = self.expression()

    self.consume(
        TokenType.RPAREN,
        "Esperado ')".
```

```

    )

    return SetExpression(
        name.lexeme,
        value
    )

def begin_expression(self):

    expressions = []

    while (
        not self.check(TokenType.RPAREN)
        and not self.is_at_end()
    ):
        expressions.append(
            self.expression()
        )

    self.consume(
        TokenType.RPAREN,
        "Esperado ')'"
    )

    return BeginExpression(
        expressions
    )

def if_expression(self):

    condition = self.expression()

    then_branch = self.expression()

    else_branch = self.expression()

    self.consume(
        TokenType.RPAREN,
        "Esperado ')'"
    )

    return IfExpression(
        condition,
        then_branch,
        else_branch
    )

```

```
def while_expression(self):

    condition = self.expression()

    body = self.expression()

    self.consume(
        TokenType.RPAREN,
        "Esperado ')'"
    )

    return WhileExpression(
        condition,
        body
    )

def peek(self):
    return self.tokens[self.current]

def advance(self):

    token = self.tokens[self.current]

    if not self.is_at_end():
        self.current += 1

    return token

def check(self, token_type):

    if self.is_at_end():
        return False

    return self.peek().type == token_type

def consume(self, token_type, message):

    if self.check(token_type):
        return self.advance()

    raise Exception(
        message
    )

def is_at_end(self):

    return self.peek().type == TokenType.EOF
```

Lexer.py

```
class TokenType(Enum):
    LPAREN = auto()
    RPAREN = auto()

    PLUS = auto()
    MINUS = auto()
    MULTIPLY = auto()
    DIVIDE = auto()

    GREATER_EQUAL = auto()
    GREATER_THAN = auto()
    LESS_EQUAL = auto()
    LESS_THAN = auto()

    NUMBER = auto()
    IDENTIFIER = auto()

    EOF = auto()

class Token:
    def __init__(self, type, lexeme, line, column):
        self.type = type
        self.lexeme = lexeme
        self.line = line
        self.column = column

    def __repr__(self):
        return f"Token({self.type.name}, {self.lexeme!r})"

class Lexer:
    def __init__(self, source):
        self.source = source

        self.start = 0
        self.current = 0

        self.line = 1
        self.column = 1

        self.tokens = []

    def tokenize(self):
        while not self.is_at_end():
            self.start = self.current
            self.scan_token()
```

```
self.tokens.append(
    Token(TokenType.EOF, "", self.line, self.column)
)

return self.tokens

def scan_token(self):
    c = self.advance()

    if c == '(':
        self.add_token(TokenType.LPAREN)

    elif c == ')':
        self.add_token(TokenType.RPAREN)

    elif c == '+':
        self.add_token(TokenType.PLUS)

    elif c == '-':
        self.add_token(TokenType.MINUS)

    elif c == '*':
        self.add_token(TokenType.MULTIPLY)

    elif c == '/':
        self.add_token(TokenType.DIVIDE)

    elif c.isspace():
        pass

    elif c.isdigit():
        self.number()

    elif c.isalpha():
        self.identifier()

    elif c == '>':
        if self.peek() == '=':
            self.advance()
            self.add_token(TokenType.GREATER_EQUAL)
        else:
            self.add_token(TokenType.GREATER_THAN)

    elif c == '<':
        if self.peek() == '=':
            self.advance()
```



```

        self.add_token(TokenType.LESS_EQUAL)
    else:
        self.add_token(TokenType.LESS_THAN)

    else:
        raise Exception(
            f"Caractere inesperado '{c}' "
            f"na linha {self.line}, coluna {self.column}"
        )

def advance(self):
    c = self.source[self.current]
    self.current += 1

    if c == '\n':
        self.line += 1
        self.column = 1
    else:
        self.column += 1

    return c

def peek(self):
    if self.is_at_end():
        return '\0'

    return self.source[self.current]

def number(self):
    while self.peek().isdigit():
        self.advance()

    self.add_token(TokenType.NUMBER)

def identifier(self):
    while self.peek().isalnum() or self.peek() == '_':
        self.advance()

    self.add_token(TokenType.IDENTIFIER)

def add_token(self, type):
    lexeme = self.source[self.start:self.current]

    self.tokens.append(
        Token(
            type,
            lexeme,

```

```
        self.line,  
        self.column  
    )  
)
```

```
def is_at_end(self):  
    return self.current >= len(self.source)
```

codegen.py

```
class MEPA:  
    def __init__(self, symbol_table):  
  
        self.symbol_table = symbol_table  
  
        self.code = []  
        self.label_count = 0  
  
    def emit(self, instruction):  
        self.code.append(instruction)  
  
    def new_label(self):  
        label = f"L{self.label_count}"  
        self.label_count += 1  
  
        return label  
  
    def emit_label(self, label):  
        self.emit(f"{label}:")  
  
    def generate(self, expressions):  
  
        for expression in expressions:  
            self.generate_expression(expression)  
  
        return self.code  
  
    def generate_expression(self, expression):  
  
        if isinstance(expression, NumberExpression):  
            return self.generate_number(expression)
```

```

    if isinstance(expression, IdentifierExpression):
        return self.generate_identifier(expression)

    if isinstance(expression, BinaryExpression):
        return self.generate_binary(expression)

    if isinstance(expression, SetExpression):
        return self.generate_set(expression)

    if isinstance(expression, PrintExpression):
        return self.generate_print(expression)

    if isinstance(expression, BeginExpression):
        return self.generate_begin(expression)

    if isinstance(expression, IfExpression):
        return self.generate_if(expression)

    if isinstance(expression, WhileExpression):
        return self.generate_while(expression)

    raise Exception(
        f"Expressão desconhecida: {expression}"
    )

    def generate_number(self, expression):

        self.emit(
            f"CRCT {expression.value}"
        )

    def generate_identifier(self, expression):

        self.emit(
            f"CRVL {expression.name}"
        )

    def generate_binary(self, expression):

```

```
self.generate_expression(  
    expression.left  
)  
  
self.generate_expression(  
    expression.right  
)  
  
operator = expression.operator  
  
if operator == "+":  
    self.emit("SOMA")  
  
elif operator == "-":  
    self.emit("SUBT")  
  
elif operator == "*":  
    self.emit("MULT")  
  
elif operator == "/":  
    self.emit("DIVI")  
  
elif operator == ">":  
    self.emit("CMMA")  
  
elif operator == ">=":  
    self.emit("CMAG")  
  
elif operator == "<":  
    self.emit("CMME")  
  
elif operator == "<=":  
    self.emit("CMEG")  
  
else:  
    raise Exception(  
        f"Operador desconhecido: {operator}"  
    )
```

```

def generate_set(self, expression):

    self.generate_expression(
        expression.value
    )

    self.emit(
        f"ARMZ {expression.name}"
    )
|
|
|
def generate_print(self, expression):

    self.generate_expression(
        expression.value
    )

    self.emit("IMPR")

def generate_begin(self, expression):

    for statement in expression.expressions:

        self.generate_expression(
            statement
        )

def generate_if(self, expression):

    else_label = self.new_label()
    end_label = self.new_label()

    self.generate_expression(
        expression.condition
    )

    self.emit(

```

```

        f"DSVF {else_label}"
    )

    self.generate_expression(
        expression.then_branch
    )

    self.emit(
        f"DSVS {end_label}"
    )

    self.emit_label(
        else_label
    )

    self.generate_expression(
        expression.else_branch
    )

    self.emit_label(
        end_label
    )

def generate_while(self, expression):

    start_label = self.new_label()
    end_label = self.new_label()

    self.emit_label(
        start_label
    )

    self.generate_expression(
        expression.condition
    )

    self.emit(
        f"DSVF {end_label}"
    )

```

```

        self.generate_expression(
            expression.body
        )

        self.emit(
            f"DSVS {start_label}"
        )

        self.emit_label(
            end_label
        )

```

report.py

```

class Report:

    @staticmethod
    def print_tokens(tokens):

        print()
        print("=" * 60)
        print("ANÁLISE LÉXICA / SINTÁTICA")
        print("=" * 60)

        print(
            f"{'NUM':<6}"
            f"{'TOKEN':<20}"
            f"{'LEXEMA'}"
        )

        print("-" * 60)

        for num, token in enumerate(tokens, start=1):

            print(
                f"{num:<6}"
                f"{token.type.name:<20}"
                f"{token.lexeme}"
            )

```

```

    )

    @staticmethod
    def print_symbols(symbol_table):

        print()
        print("=" * 60)
        print("TABELA DE SÍMBOLOS")
        print("=" * 60)

        print(
            f"{'Identificador':<18}"
            f"{'Endereço MEPA':<18}"
            f"{'Tipo Presumido':<18}"
            f"{'Escopo'}"
        )

        print("-" * 75)

        for symbol in symbol_table.symbols.values():

            print(
                f"{symbol.name:<18}"
                f"{symbol.address:<18}"
                f"{symbol.type:<18}"
                f"{symbol.scope}"
            )

    @staticmethod
    def print_code(code):

        print()
        print("=" * 60)
        print("GERAÇÃO DE CÓDIGO INTERMEDIÁRIO")
        print("=" * 60)

        for instruction in code:
            print(instruction)

```


expressions.py

```
class NumberExpression:
    def __init__(self, value):
        self.value = value

    def __repr__(self):
        return f"Number({self.value})"

    def dump(self, indent=0):
        return " " * indent + f"Number({self.value})"

class IdentifierExpression:
    def __init__(self, name):
        self.name = name

    def __repr__(self):
        return f"Identifier({self.name})"

    def dump(self, indent=0):
        return " " * indent + f"Identifier({self.name})"

class BinaryExpression:
    def __init__(self, operator, left, right):
        self.operator = operator
        self.left = left
        self.right = right

    def __repr__(self):
        return f"Binary({self.operator}, {self.left}, {self.right})"

    def dump(self, indent=0):
        result = " " * indent + f"Binary({self.operator})\n"

        result += self.left.dump(indent + 2)
        result += "\n"
        result += self.right.dump(indent + 2)
```

```
    return result
```

```
class SetExpression:
```

```
    def __init__(self, name, value):
```

```
        self.name = name
```

```
        self.value = value
```

```
    def __repr__(self):
```

```
        return f"Set({self.name}, {self.value})"
```

```
    def dump(self, indent=0):
```

```
        result = " " * indent + f"Set({self.name})\n"
```

```
        result += self.value.dump(indent + 2)
```

```
    return result
```

```
class PrintExpression:
```

```
    def __init__(self, value):
```

```
        self.value = value
```

```
    def __repr__(self):
```

```
        return f"Print({self.value})"
```

```
    def dump(self, indent=0):
```

```
        result = " " * indent + "Print\n"
```

```
        result += self.value.dump(indent + 2)
```

```
    return result
```

```
class BeginExpression:
```

```
    def __init__(self, expressions):
```

```
        self.expressions = expressions
```

```
    def __repr__(self):
```

```
        return f"Begin({self.expressions})"
```

```

def dump(self, indent=0):
    result = " " * indent + "Begin\n"

    for expression in self.expressions:
        result += expression.dump(indent + 2)
        result += "\n"

    return result.rstrip()

```

```

class IfExpression:
    def __init__(
        self,
        condition,
        then_branch,
        else_branch
    ):
        self.condition = condition
        self.then_branch = then_branch
        self.else_branch = else_branch

```

```

    def __repr__(self):
        return (
            f"If("
            f"condition={self.condition}, "
            f"then={self.then_branch}, "
            f"else={self.else_branch}"
            f")"
        )

```

```

    def dump(self, indent=0):
        result = " " * indent + "If\n"

        result += " " * (indent + 2) + "Condition\n"
        result += self.condition.dump(indent + 4)
        result += "\n"

        result += " " * (indent + 2) + "Then\n"

```

```
    result += self.then_branch.dump(indent + 4)
    result += "\n"
```

```
    result += " " * (indent + 2) + "Else\n"
    result += self.else_branch.dump(indent + 4)
```

```
    return result
```

```
class WhileExpression:
```

```
    def __init__(self, condition, body):
        self.condition = condition
        self.body = body
```

```
    def __repr__(self):
```

```
        return (
            f"While("
            f"condition={self.condition}, "
            f"body={self.body}"
            f")"
        )
```

```
    def dump(self, indent=0):
```

```
        result = " " * indent + "While\n"
```

```
        result += " " * (indent + 2) + "Condition\n"
        result += self.condition.dump(indent + 4)
        result += "\n"
```

```
        result += " " * (indent + 2) + "Body\n"
        result += self.body.dump(indent + 4)
```

```
    return result
```

test_semantic.py

```
def test_semantic():
```

```
    _source = """
```

```

(begin
  (set a 10)
  (set b 20)

  (print (+ a b))

  (if (>= a b)
    (print a)
    (print b)
  )

  (while (< a 100)
    (begin
      (print a)
      (set a (+ a 1))
    )
  )

)
"""

lexer = Lexer(source)
tokens = lexer.tokenize()

parser = Parser(tokens)
expressions = parser.parse()

semantic_analyzer = SemanticAnalyzer()

semantic_analyzer.analyze(
  expressions
)

print()
print("===== TESTE DO SEMÂNTICO =====")

print("Análise semântica concluída!")

```

```

print()
print("Tabela de símbolos:")

for symbol in semantic_analyzer.symbol_table.symbols.values():

    print(
        f"{symbol.name:<15}"
        f"{symbol.address:<10}"
        f"{symbol.type:<15}"
        f"{symbol.scope}"
    )

print("=====")
|
|
|
def test_semantic_negative():

    source = """
    (print x)
    """

    lexer = Lexer(source)
    tokens = lexer.tokenize()

    parser = Parser(tokens)
    expressions = parser.parse()

    semantic_analyzer = SemanticAnalyzer()

    print()
    print("===== TESTE NEGATIVO DO SEMÂNTICO =====")

    try:

        semantic_analyzer.analyze(
            expressions
        )

```

```
    print(
        "ERRO: o Semântico deveria "
        "rejeitar a variável 'x'."
    )

except Exception as e:

    print("Erro detectado corretamente:")
    print(e)
```

test_parser.py

```
def test_parser():

    source = """
    (begin
      (set a 10)
      (set b 20)

      (print (+ a b))

      (if (>= a b)
        (print a)
        (print b)
      )

      (while (< a 100)
        (set a (+ a 1))
      )
    )
    """

    lexer = Lexer(source)

    tokens = lexer.tokenize()

    parser = Parser(tokens)

    expressions = parser.parse()
```

```

print()
print("==== TESTE DO PARSER =====")

for expression in expressions:
    print(expression.dump())

print("=====")
|
|
|
def test_parser_negative():

    source = """
    (print 10 20)
    """

    lexer = Lexer(source)
    tokens = lexer.tokenize()

    parser = Parser(tokens)

    print()
    print("==== TESTE NEGATIVO DO PARSER =====")

    try:

        parser.parse()

        print(
            "ERRO: o Parser deveria rejeitar "
            "(print 10 20). "
        )

    except Exception as e:

        print("Erro detectado corretamente:")
        print(e)

```


test_lexer.py

```
def test_lexer():

    source = """
    (begin
      (set idade 20)
      (set contador 1)

      (if (>= idade 18)
        (print 1)
        (print 0)
      )

      (while (< contador 10)
        (begin
          (print contador)
          (set contador (+ contador 1))
        )
      )
    )
    """

    lexer = Lexer(source)

    tokens = lexer.tokenize()

    print()
    print("==== TESTE DO LEXER =====")

    for num, token in enumerate(tokens, start=1):
        print(
            f"{num:<4} "
            f"{token.type.name:<20} "
            f"{token.lexeme}"
        )

    print("=====")
```

```

def test_lexer_negative():
    source = """
    (print @)
    """

    lexer = Lexer(source)

    print()
    print("===== TESTE NEGATIVO DO LEXER =====")

    try:
        lexer.tokenize()

        print("ERRO: o Lexer deveria rejeitar '@'.")

    except Exception as e:

        print("Erro detectado corretamente:")
        print(e)

```

test_compiler.py

```

def test_compiler(source):

    print()
    print("=" * 70)
    print("TESTE DE INTEGRAÇÃO")
    print("=" * 70)

    print()
    print("CÓDIGO FONTE")
    print("-" * 70)
    print(source)

    lexer = Lexer(source)

    tokens = lexer.tokenize()

```

```
print()
Report.print_tokens(tokens)

parser = Parser(tokens)

expressions = parser.parse()

print()
print("=" * 70)
print("ÁRVORE SINTÁTICA ABSTRATA")
print("=" * 70)

for expression in expressions:
    print(expression.dump())

semantic_analyzer = SemanticAnalyzer()

semantic_analyzer.analyze(
    expressions
)

Report.print_symbols(
    semantic_analyzer.symbol_table
)

generator = MEPA(
    semantic_analyzer.symbol_table
)

code = generator.generate(
    expressions
)

Report.print_code(code)

print()
print("=" * 70)
print("COMPILAÇÃO CONCLUÍDA COM SUCESSO")
```

```
print("=" * 70)
```